

Apache Airflow Case Study Documentation

1. Introduction

Apache Airflow is an open-source platform to programmatically author, schedule, and monitor workflows. It allows building complex data pipelines using Directed Acyclic Graphs (DAGs) and provides a web interface to manage and monitor workflow execution.

2. Key Features of Airflow

- **DAG-based Workflows:** Pipelines are defined as Directed Acyclic Graphs for better visualization and management.
 - **Dynamic Pipeline Generation:** Workflows are written in Python, enabling dynamic creation of tasks.
 - **Task Scheduling:** Provides robust scheduling capabilities with cron-like expressions.
 - **Scalability:** Can scale horizontally across multiple workers for handling large workloads.
 - **Extensible:** Supports custom operators, sensors, and hooks to integrate with external systems.
 - **Monitoring and Logging:** Built-in web interface for monitoring DAG runs, task logs, and execution status.
 - **Retry and Alerting:** Configurable retries, alerts, and error handling for reliable execution.
 - **Integration:** Works with multiple data sources, cloud services, and orchestration tools.
-

3. Building a Pipeline in Airflow

To build and run a pipeline in Airflow, follow these steps:

Step 1: Define a DAG File

1. Open the Airflow project directory.
2. Locate the dags folder.
3. Create a new Python file (e.g., example_dag.py).

4. Import the required Airflow libraries such as DAG, operators, and datetime.
 5. Define the default arguments like owner, start date, retries, etc.
 6. Initialize a DAG object with a unique DAG ID and scheduling interval.
 7. Define tasks using available operators (e.g., BashOperator, PythonOperator).
 8. Set dependencies between tasks using >> or <<.
-

Step 2: Place the DAG in the Correct Folder

- Save the Python file in the dags directory.
 - Ensure the file has no syntax errors, as Airflow scans this folder to detect DAGs.
-

Step 3: Start Airflow Services

In project folder (where docker-compose.yml is), start service using this command

`docker-compose up -d`

Check that containers are running

`docker ps`

1. Start the Airflow web server.
 2. Start the Airflow scheduler.
 3. Ensure both services are running in the background.
-

Step 4: Access Airflow UI

1. Open a browser and go to the Airflow web interface (default: `http://localhost:8080`).
 2. Log in using the default or configured credentials.
 3. Navigate to the **DAGs page**, where all available pipelines will be listed.
-

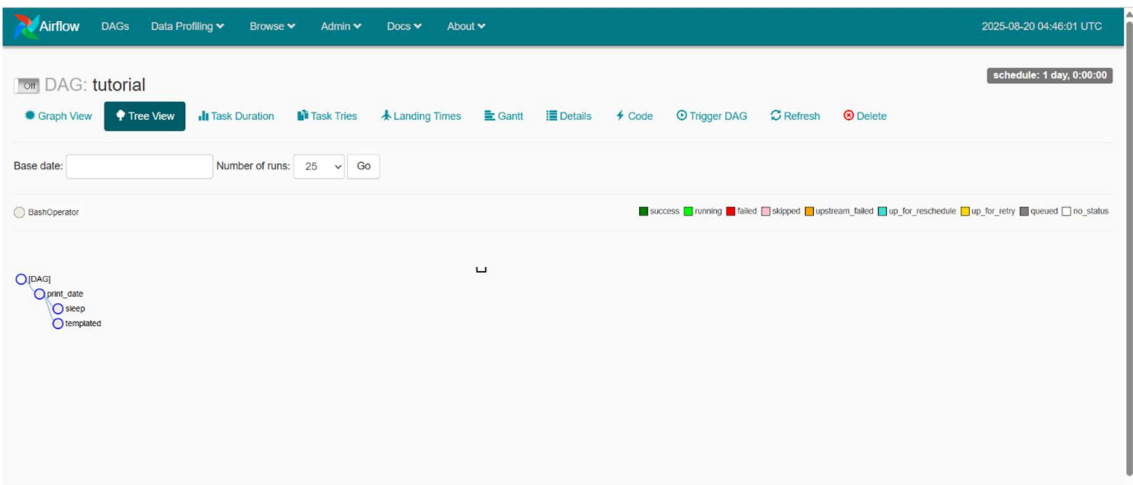
Step 5: Enable and Run a DAG

1. Locate the newly created DAG in the list.
2. Toggle the switch to enable the DAG.
3. Trigger the DAG manually or wait for the scheduled run.

Step 6: Monitor Pipeline Execution

1. Click on the DAG name to view details.
2. Use the **Graph View** to visualize task dependencies.
3. Use the **Tree View** to track execution history.
4. Open **Task Logs** to view real-time logs of each task execution.

Sample Screenshot of Tree View:



4. Viewing and Managing Pipelines

- **DAGs List View:** Shows all available workflows with their current status.
 - **Graph View:** Visualizes the structure of the DAG and task dependencies.
 - **Tree View:** Displays task run history over multiple DAG runs.
 - **Task Instance Details:** Provides logs, retry options, and execution information.
 - **Code View:** Displays the underlying DAG Python code directly in the UI.
-

5. Best Practices

- Use clear and descriptive DAG and task names.
 - Keep DAG files lightweight and modular.
 - Define retries and failure handling mechanisms.
 - Use variables and connections for configurable settings.
-